GoPLC and the Humanoid Robot Control Problem

A Technical Architecture Analysis: Applying Industrial PLC Runtime Principles to Next-Generation Humanoid Robotics

James Belcher | JMB Technical Services | GoPLC Project | 2026

Version 1.1 — Technical Whitepaper (Updated March 2026)

Abstract

Humanoid robots represent one of the most demanding real-time control problems in modern engineering: dozens of actuators operating at sub-millisecond scan rates, heterogeneous communication buses, hard safety requirements, and increasingly, tight integration with AI inference systems. This paper examines how GoPLC — a full-featured industrial PLC runtime written in Go — maps to this problem domain. Using Tesla's Optimus Gen 2 as a concrete reference platform, we derive a 12-runtime clustered architecture that satisfies both the real-time determinism requirements of low-level actuator control and the high-level AI orchestration demands of an autonomous robot. We argue that the industrial automation field has already solved the hard problems of multi-task scheduling, protocol abstraction, and safety isolation, and that the robotics industry stands to benefit substantially from adopting these proven patterns.

Contents

1. Introduction: Why Robots Need PLCs
2. The Optimus Gen 2 Hardware Problem
3. GoPLC Architecture Overview
4. Runtime Decomposition and Assignment
5. Clustered Node Layout
6. Scan Cycle Analysis and Task Scheduling
7. HAL Layer and Protocol Drivers
8. Safety Architecture and SIL Compliance
9. AI-Native Integration: The MCP Server Bridge
10. Structured Text Across the Stack
11. Comparison with Existing Approaches
12. Conclusions and Future Work
13. References

---

1. Introduction: Why Robots Need PLCs

The industrial automation industry spent decades solving a class of problems that robotics engineers are now confronting from scratch: how do you reliably coordinate dozens of heterogeneous I/O points, each with different timing requirements, across a distributed computing substrate, while maintaining hard real-time guarantees and achieving functional safety certification?

Programmable Logic Controllers (PLCs) emerged as the canonical answer to this question in process automation, and the IEC 61131-3 standard [12] codified the execution model that now governs billions of dollars of installed industrial infrastructure. The scan-cycle model, task prioritization, watchdog timers, and structured programming languages that define modern PLC runtimes are not arbitrary conventions — they are hard-won engineering solutions to exactly the problems that humanoid robots face.

GoPLC is a software PLC runtime that implements the IEC 61131-3 execution model in the Go programming language. It provides multi-task scheduling with configurable scan rates, a hardware abstraction layer (HAL) supporting eighteen protocol drivers, distributed clustering, and — critically — a native Model Context Protocol (MCP) server interface that enables AI systems to interact with the runtime as a first-class tool. This paper explores how GoPLC's architecture maps onto the control problem posed by a state-of-the-art humanoid robot.

Key Insight: The industrial automation field solved real-time distributed control decades ago. Humanoid robotics is reinventing the wheel — GoPLC bridges that gap.

2. The Optimus Gen 2 Hardware Problem

Tesla's Optimus Gen 2 serves as a useful reference platform because it is among the most mechanically sophisticated humanoid robots in production and its high-level specifications are publicly known. The challenge it presents to a control system is representative of the broader humanoid robotics class.

2.1 Degrees of Freedom

Optimus Gen 2 implements 28 primary degrees of freedom across the body using 28 custom-designed actuators — 14 rotary (frameless torque motor + harmonic drive) and 14 linear (frameless torque motor + planetary roller screw) [1][2]. The hands contribute 11 DOF each through individual finger actuation. This DOF distribution creates sharply divergent control requirements:

Subsystem	DOF	Actuator Type	Nominal Scan Rate	Notes
Left Hand	11	Linear + rotary	1–2 ms	Tactile sensors per finger
Right Hand	11	Linear + rotary	1–2 ms	Mirror of left
Left Arm	6	Rotary servo	2–4 ms	Shoulder, elbow, wrist
Right Arm	6	Rotary servo	2–4 ms	Mirror of left
Left Leg	6	Rotary servo	2–4 ms	Hip, knee, ankle, foot
Right Leg	6	Rotary servo	2–4 ms	Mirror of left
Torso/Spine	3	Rotary servo	5 ms	Load-bearing, high torque
Head/Neck	3	Rotary servo	10 ms	Camera/LIDAR mount
Total	**52+**	Mixed	1–10 ms	Excl. sensor-only axes

Table 1: Optimus Gen 2 estimated DOF distribution

2.2 The Scan Rate Heterogeneity Problem

The single most challenging aspect of humanoid robot control is the ten-to-one span in required scan rates across subsystems. Finger force feedback loops require 1 ms or faster to achieve stable contact control. Gait planning operates at 20–50 ms. Battery management is comfortable at 100 ms. AI inference calls may take 50–500 ms.

A naive single-threaded control loop cannot satisfy all of these simultaneously. The standard industrial solution — multi-task scheduling with priority-assigned scan cycles — is precisely what GoPLC implements. This is not a coincidence; it is the correct architectural response to this class of problem.

2.3 Communication Bus Heterogeneity

Humanoid robots incorporate multiple physical communication buses optimized for different subsystems. High-performance servo drives typically use EtherCAT for its deterministic sub-millisecond cycle times. Battery management systems commonly use CAN bus. Sensor arrays may use SPI, I2C, or proprietary serial protocols. Any viable control architecture must abstract these differences behind a common programming model — exactly what GoPLC's HAL layer provides.

2.4 Compute Architecture

The Optimus compute platform shares lineage with the Full Self-Driving (FSD) hardware used in Tesla vehicles [3]. The current generation AI4 chip employs a dual-SoC configuration running two 20-core computers (ARM Cortex-A72, 2.35 GHz) in parallel, constantly cross-checking each other [4][5]. This redundant core topology is borrowed directly from automotive functional safety design, where paired cores execute identical computations and cross-check results to detect hardware faults. If one SoC fails, the other takes over instantly — a fundamental requirement for advancing toward higher levels of autonomous operation [4].

This architecture has direct implications for control system design:

Parameter	Value	Implication
SoC count	2 (dual redundant)	Natural fault domain boundary [4]
Total cores	20 (ARM Cortex-A72)	10 usable cores after redundancy pairing [5]
Core frequency	2.35 GHz	Up from 12 cores in previous generation [5]
Core configuration	Redundant/lockstep	Hardware-level fault detection (ASIL-D heritage) [4]
Memory bandwidth	384 GB/s (GDDR6)	High-throughput sensor and video processing [5]
Shared with	Tesla FSD (vehicles)	Mature silicon, proven at scale [3]
Neural accelerator	Yes (integrated)	On-device AI inference without external GPU [5]

Table 1b: Optimus Gen 2 compute architecture (publicly disclosed)

The dual-SoC design provides a natural physical boundary for runtime partitioning. With 10 usable cores available after redundancy pairing, the 12-runtime architecture proposed in Section 4 fits within the compute budget — most runtimes require well under a full core, and the safety monitor (RT-10) can be mapped to a dedicated redundant core pair for hardware-backed fault isolation.

The integrated neural accelerator is particularly significant for the AI bridge runtime (RT-12). On-device inference eliminates the round-trip latency to cloud-based LLMs for time-sensitive decisions, while the MCP interface can still route to cloud models for complex reasoning tasks that tolerate higher latency.

2.5 Next-Generation Compute: AI5 and Optimus Gen 3

Tesla's next-generation AI5 chip, part of the Hardware 5 (HW5) platform, represents a generational leap in onboard compute [6]. Publicly disclosed specifications suggest:

Parameter	AI4 (Current)	AI5 (Next Gen)	Improvement
Raw compute	Baseline	~8x	Order of magnitude [6]
Inference performance	Baseline	~40–50x	Architectural + process gains [7]
Memory capacity	Baseline	~9x	Larger models, more state [6]
Memory bandwidth	Baseline	~5x	Reduced inference bottleneck [6]
Power efficiency	Baseline	~3x per watt	Critical for battery-powered robot [7]
Process node	—	2 nm	TSMC and Samsung fabrication [8]

Table 1c: AI5 performance projections relative to AI4 (publicly disclosed estimates)

AI5 production is expected to begin in late 2026, with volume manufacturing in mid-2027 [6]. The chips will be fabricated by both TSMC and Samsung [8]. Tesla's Terafab facility in Austin — a $20 billion vertically integrated chip fabrication project launched March 21, 2026 — will produce AI5 at scale, targeting 100,000 wafer starts per month and 100–200 billion AI and memory chips annually [9][10].

Optimus Gen 3, slated for summer 2026 production, expands the mechanical platform to approximately **50 actuators** (nearly doubling Gen 2) [11]. The upgraded hands feature 22 DOF per hand (up from 11), enabled by relocating all 25 actuators per side into the forearm using a tendon-driven architecture [2][11]. Gen 3 also integrates Grok — xAI's large language model — for natural language interaction, enabling the robot to receive verbal instructions, ask clarifying questions, and adapt in real time [11]. The combination of doubled actuator

count and 40–50x more compute fundamentally changes the control problem:

- **More actuators demand more runtimes.** The 12-runtime architecture scales naturally — additional body subsystems (individual finger joints, facial expression actuators, additional sensor arrays) map to additional runtimes within the existing cluster model.
- **50x compute headroom enables richer control loops.** Force feedback algorithms can incorporate more sensor fusion, predictive models can run at servo rate, and the coordinator runtime (RT-11) can execute more sophisticated motion planning without stealing cycles from limb control.
- **On-device LLM inference becomes practical.** With 9x memory and 40x inference performance, running a capable language model directly on the robot — rather than relying on cloud connectivity — becomes viable. RT-12's MCP bridge can route between on-device Grok for real-time interaction and cloud-based models for complex planning.
- **3x power efficiency extends operational time.** For a battery-powered humanoid, compute power budget is a direct constraint on mission duration. AI5's efficiency gains mean more capable control without proportional battery impact.

Forward-Looking Implication: The GoPLC architecture proposed in this paper is designed for Gen 2 constraints. On AI5 hardware, the same architecture runs with an order-of-magnitude headroom — enough to add runtimes, increase scan rates, and run on-device AI inference simultaneously. The architecture scales up without redesign.

3. GoPLC Architecture Overview

GoPLC implements the IEC 61131-3 execution model as a native Go application. The runtime compiles and executes Structured Text (ST) programs, manages task scheduling, handles I/O through protocol drivers, and exposes control surfaces through REST API and MCP server interfaces.

3.1 Multi-Task Scheduling

Each GoPLC runtime instance supports multiple concurrent tasks with independently configured scan periods and priorities. Tasks share a common variable space and can exchange data through the internal symbol table. The scheduler uses Go's goroutine model with real-time priority hints to achieve deterministic timing. A single runtime can host tasks with scan rates spanning three orders of magnitude without interference, provided CPU budget is managed appropriately.

3.2 Protocol Drivers and HAL

GoPLC ships with eighteen built-in protocol drivers covering the major industrial communication standards: Modbus TCP/RTU, EtherNet/IP (CIP), OPC UA, S7 (Siemens), DNP3, BACnet/IP, FINS (Omron), IEC 60870-5-104, SNMP, MQTT, and hardware HALs for Propeller 2, Arduino, and ctrlX EtherCAT. The HAL layer abstracts these behind a unified tag-based I/O model. From the perspective of Structured Text programs, all I/O is memory-mapped symbols — the underlying protocol is an implementation detail. This abstraction is critical for robotics applications where the servo bus protocol may change between hardware generations.

3.3 Clustering

Multiple GoPLC instances can be linked into a cluster, sharing variable namespaces and synchronizing state across nodes. This enables runtime distribution across multiple compute nodes while maintaining a unified programming model. For humanoid robotics, clustering maps naturally to a multi-SoC architecture where different compute nodes handle different body regions.

3.4 REST API and MCP Server

GoPLC exposes 250+ REST API endpoints covering the full runtime lifecycle: program compilation and deployment, variable read/write, task management, diagnostic data, and configuration. The MCP server wraps this API in the Model Context Protocol through 32 structured tools across 7 modules, enabling AI systems including Claude to interact with the runtime as a first-class tool. Additionally, the runtime includes a built-in AI assistant with an agentic control loop — the AI can autonomously read variables, write setpoints, deploy programs, and manage tasks through 12 built-in control tools, all accessible from the web IDE's chat interface.

4. Runtime Decomposition and Assignment

Given the hardware analysis in Section 2 and the GoPLC capabilities in Section 3, we can derive a principled runtime decomposition for a humanoid robot of the Optimus class. The decomposition follows three primary criteria: scan rate compatibility, fault isolation boundary, and functional cohesion.

4.1 Decomposition Principles

- **Scan rate grouping:** Tasks with similar scan rate requirements share a runtime. A 1 ms hand control runtime should not share CPU budget with a 100 ms battery monitor.
- **Fault isolation:** Safety-critical functions must be in dedicated runtimes that cannot be starved or corrupted by faults in adjacent domains.
- **Bilateral symmetry:** Left and right limb pairs get independent runtimes. This provides fault isolation (right arm failure does not affect left arm) and enables parallel development and testing.
- **Physical co-location:** Runtimes that share a physical bus or compute node are grouped to minimize inter-runtime communication latency.

4.2 The Twelve-Runtime Architecture

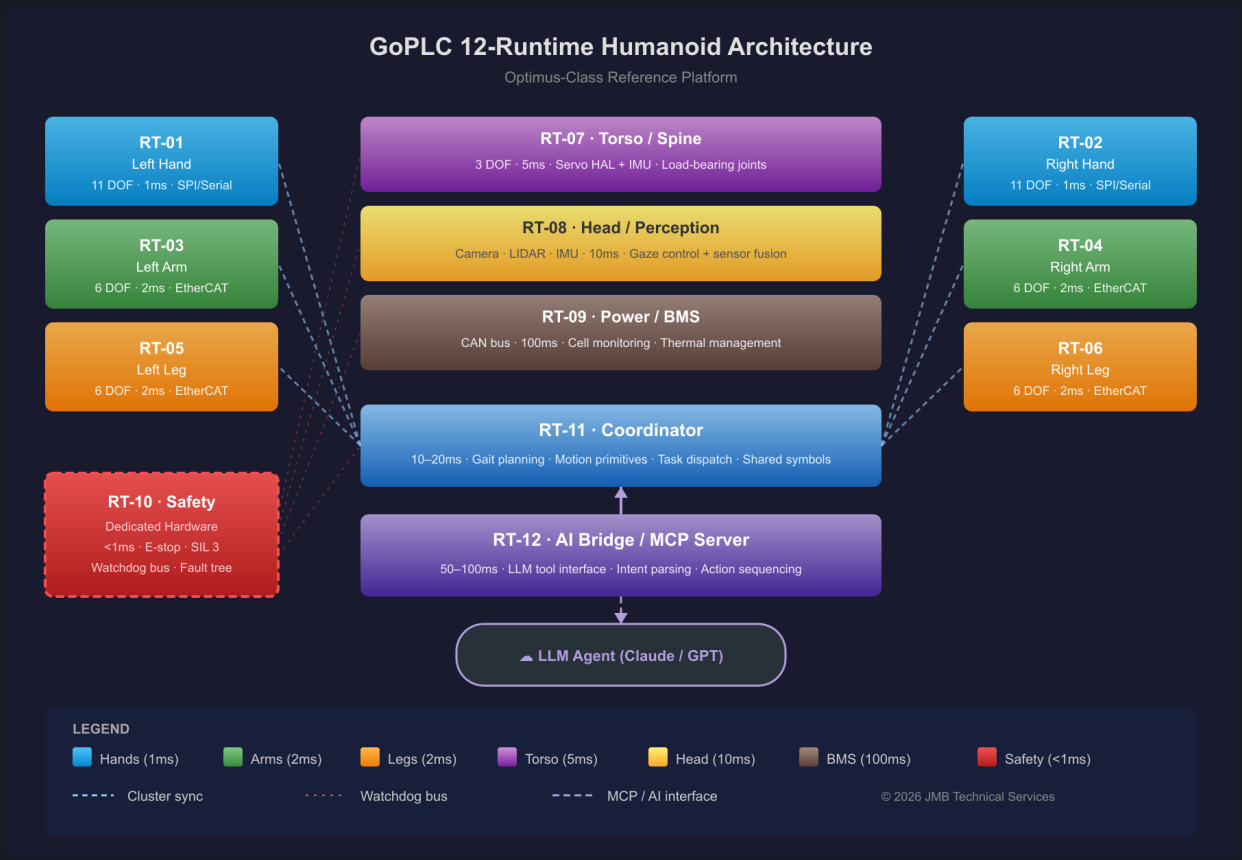

Figure 1: Complete 12-runtime architecture showing bilateral symmetry, coordinator, AI bridge, and isolated safety monitor

RT#	Domain	Protocols / HAL	Scan Cycle	Key Functions
RT-01	Left Hand	SPI/Serial HAL, tactile array	1–2 ms	11 DOF, force/torque per finger, grip control
RT-02	Right Hand	SPI/Serial HAL, tactile array	1–2 ms	Mirror of RT-01, independent fault domain
RT-03	Left Arm	EtherCAT HAL	2–4 ms	Shoulder, elbow, wrist servo control
RT-04	Right Arm	EtherCAT HAL	2–4 ms	Mirror of RT-03
RT-05	Left Leg	EtherCAT HAL	2–4 ms	Hip, knee, ankle, foot, balance feedback
RT-06	Right Leg	EtherCAT HAL	2–4 ms	Mirror of RT-05
RT-07	Torso/Spine	Servo HAL, IMU	5 ms	Load-bearing joints, center-of-mass sensor fusion
RT-08	Head/Perception	Camera, LIDAR, IMU HAL	10 ms	Sensor preprocessing, gaze control, orientation
RT-09	Power / BMS	CAN bus HAL	100 ms	Cell monitoring, thermal management, charge control
RT-10	Safety Monitor	Dedicated watchdog bus	< 1 ms	E-stop, SIL watchdog, fault tree, emergency hold
RT-11	Coordinator	Internal IPC, shared symbols	10–20 ms	Gait planning, motion primitives, task dispatch
RT-12	AI Bridge / MCP	REST + WebSocket + MCP	50–100 ms	LLM tool interface, intent parsing, action sequencing

Table 2: Complete 12-runtime assignment for Optimus-class humanoid

Hard Constraint: RT-10 (Safety Monitor) is the only runtime that may never share a CPU core with any other runtime. This is a hard architectural constraint, not a recommendation.

4.3 Runtime Interaction Model

Runtimes interact through two mechanisms. Within a cluster node, runtimes share a symbol namespace and communicate through direct variable references at zero additional latency. Across nodes, GoPLC's clustering protocol replicates variable state at configurable frequencies. For the humanoid application, inter-node synchronization at 10 ms is sufficient for the coordinator and AI bridge runtimes, while limb runtimes operate independently within their node.

5. Clustered Node Layout

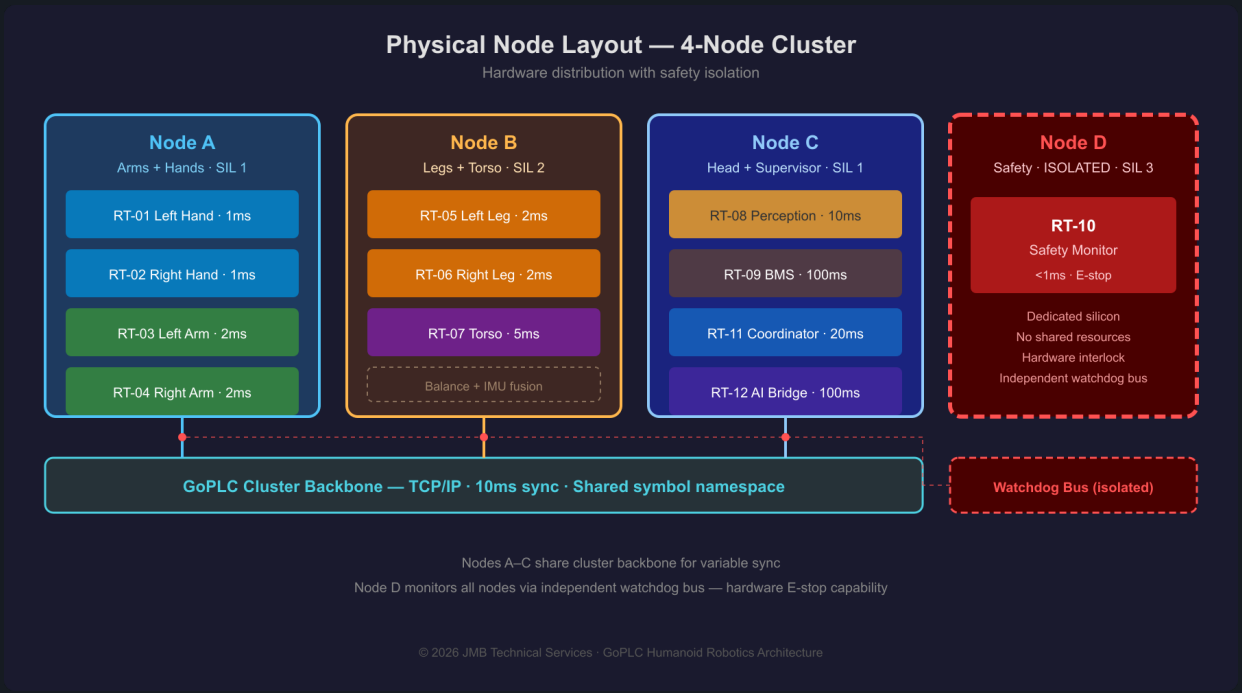

Figure 2: Physical 4-node cluster with safety isolation — Node D has no shared resources with any other node

The twelve runtimes distribute across four physical compute nodes. On the current Optimus Gen 2 dual-SoC platform (Section 2.4), this maps naturally to the available silicon: each SoC hosts a subset of runtimes, with the safety monitor on a dedicated redundant core pair. This layout is driven by physical bus topology, thermal constraints, and safety isolation requirements.

Node	Runtimes	Primary Responsibility	Safety Level
Node A (Arms + Hands)	RT-01, RT-02, RT-03, RT-04	All upper body actuation; finger force + arm servo	SIL 1
Node B (Legs + Torso)	RT-05, RT-06, RT-07	Locomotion and balance; load-bearing servo control	SIL 2
Node C (Head + Supervisor)	RT-08, RT-09, RT-11, RT-12	Perception, BMS, gait planning; AI bridge and coordination	SIL 1
Node D (Safety — Isolated)	RT-10	E-stop, watchdog, fault tree; hardware interlock control	SIL 3

Table 3: Physical node layout and safety classification

Node D warrants special attention. The safety monitor runtime runs on dedicated hardware with no shared resources. It monitors all other nodes via a separate watchdog bus and can assert a hardware E-stop independently of software state on any other node. This is the minimum architecture required to achieve SIL 2 system-level certification under IEC 62061 [13].

6. Scan Cycle Analysis and Task Scheduling

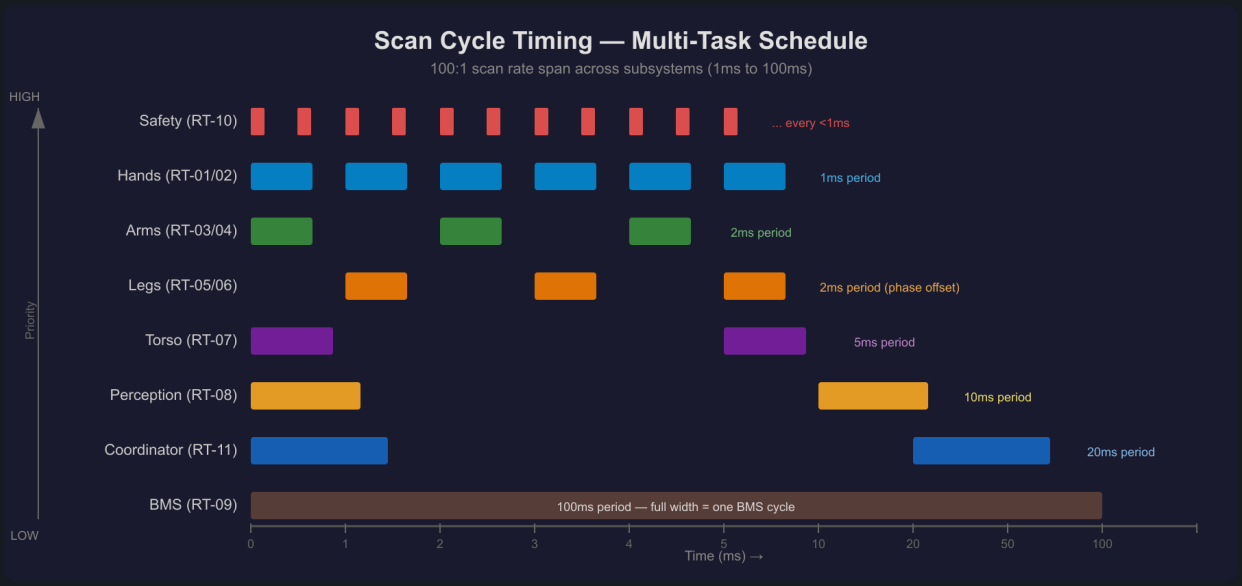

Figure 3: Multi-task timing diagram showing the 100:1 scan rate span — from sub-millisecond safety watchdog to 100ms BMS monitoring

Achieving stable 1 ms scan cycles for hand control while simultaneously running 100 ms BMS cycles in the same architecture requires careful CPU budget management. GoPLC's multi-task scheduler handles this through priority-based preemption within a runtime and OS-level isolation between runtimes.

6.1 Intra-Runtime Task Layering

Each hand runtime (RT-01, RT-02) hosts three tasks of decreasing priority:

- **Task 1 — Force Control (1 ms, Priority 1):** Inner loop force/torque feedback for each finger. Reads tactile sensor array, computes correction, writes actuator setpoint. Must never miss a cycle.
- **Task 2 — Grip Management (5 ms, Priority 2):** Executes grip primitives (pinch, power, precision), manages state transitions, handles slip detection.
- **Task 3 — Diagnostics (100 ms, Priority 3):** Logs actuator temperatures, wear counters, fault history. Non-critical, can be pre-empted freely.

This three-layer pattern repeats across all limb runtimes, scaled to their respective scan rate requirements. The IEC 61131-3 task model maps directly to this structure — Structured Text programs are assigned to tasks, not runtimes, providing fine-grained scheduling control.

6.2 CPU Budget Allocation

Runtime	Scan Rate	Est. CPU% (single core)	Jitter Budget
RT-01/02 (Hands)	1 ms	15–20% each	< 50 µs
RT-03/04 (Arms)	2 ms	8–12% each	< 100 µs
RT-05/06 (Legs)	2 ms	8–12% each	< 100 µs
RT-07 (Torso)	5 ms	4–6%	< 500 µs
RT-08 (Perception)	10 ms	10–15%	< 1 ms
RT-09 (BMS)	100 ms	1–2%	< 10 ms
RT-10 (Safety)	< 1 ms	5–8% dedicated	< 10 µs
RT-11 (Coordinator)	10–20 ms	5–10%	< 2 ms
RT-12 (AI Bridge)	50–100 ms	3–5%	Non-critical

Table 4: Estimated CPU budget per runtime

6.3 Measured Performance Data

The CPU budgets and jitter targets in Table 4 are not speculative — they are grounded in extensive benchmarking of the GoPLC runtime across multiple configurations. The following data was measured on production hardware running GoPLC v1.0.186+ on AMD 24-core/32-thread systems.

Sub-Millisecond Scan Cycle Stability

GoPLC achieves stable sub-millisecond scan cycles with near-linear scaling across runtime instances:

Scan Target	Single Runtime	31 Runtimes	Scaling Efficiency
1 ms	974 scans/s	30,258 scans/s	100.0%
500 µs	2,000 scans/s	62,336 scans/s	100.5%
250 µs	4,003 scans/s	124,186 scans/s	100.1%
100 µs	10,007 scans/s	310,586 scans/s	100.1%
50 µs	**20,012 scans/s**	**620,949 scans/s**	**100.1%**

Table 4b: Measured sub-millisecond scaling (PID loop workload, AMD 24-core) [18]

At 50 µs scan targets — 20x faster than the hand control loops proposed for RT-01/02 — GoPLC maintains 100.1% efficiency across 31 concurrent runtimes. The 12-runtime humanoid architecture operates well within the validated performance envelope.

PREEMPT_RT Linux Jitter

On PREEMPT_RT kernels with SCHED_FIFO scheduling and CPU pinning, GoPLC achieves hard real-time jitter characteristics:

Configuration	Scan Rate	Measured Jitter
PREEMPT_RT + SCHED_FIFO + CPU pin	100 µs	**< 10 µs**
PREEMPT_RT + container + CPU pin	100 µs	< 100 µs
Standard kernel, RT mode	1 ms	< 300 µs (p99)
Standard kernel, no RT mode	1 ms	< 970 µs (p99)

Table 4c: Measured jitter by kernel and scheduling configuration [18][19]

The **< 10 µs jitter at 100 µs scan rate** on PREEMPT_RT is comparable to dedicated RTOS performance and exceeds the requirements for all runtimes in the proposed architecture, including the safety monitor (RT-10).

Real-Time Mode Impact (Standard Kernel)

Even without PREEMPT_RT, GoPLC's RT mode (`LockOSThread`, tuned GC, memory locking) provides significant jitter reduction:

Percentile	Standard Mode	RT Mode	Improvement
p50	64 µs	62 µs	1.0x
p90	124 µs	105 µs	1.2x
p95	890 µs	163 µs	**5.5x**
p99	959 µs	898 µs	1.1x

Table 4d: Sustained jitter over 60 seconds, PID loop, 1 ms scan target [18]

Runtime Density at Scale

The 12-runtime humanoid architecture is a fraction of GoPLC's demonstrated capacity. Benchmark data shows:

Runtimes	p99 Jitter (1 ms target)	Scaling Efficiency
1	961 µs	100.0%
10	963 µs	98.4%
50	1,042 µs	96.2%

Runtimes	p99 Jitter (1 ms target)	Scaling Efficiency
100	958 µs	98.2%
200	966 µs	100.3%
500	**2,120 µs**	**94.9%**

Table 4e: Runtime scaling efficiency, PID loop, 1 ms scan target [18]

At 500 concurrent runtimes with >94% efficiency, the 12-runtime humanoid architecture uses approximately 2.4% of the demonstrated scaling capacity. This margin is critical: it means the architecture can absorb additional runtimes for Gen 3's expanded actuator count without approaching system limits.

Hardware I/O Validation

GoPLC has been tested against real industrial I/O hardware at servo-rate speeds. On a Bosch ctrlX CORE (ARM Cortex-A53) with live EtherCAT bus:

Metric	Measured
EtherCAT bus cycle	2 ms (500 Hz)
GoPLC scan rate (matched)	2 ms
Average scan duration	0.69 ms
I/O update rate	500 Hz
Errors	0

Table 4f: Measured EtherCAT I/O performance, ctrlX CORE ARM platform [20]

Multi-Protocol Live Validation (ARM64)

Beyond single-protocol benchmarks, GoPLC has been validated running six industrial protocols simultaneously on ARM64 hardware (Bosch ctrlX CORE X3, Cortex-A53), with all client connections reading dynamic data from a remote server:

Protocol	Role	Scan Period	Actual Execution	CPU Idle	Avg Jitter
Modbus TCP	Client	50 ms	1.8 ms	**96.4%**	873 µs
FINS (Omron)	Client	50 ms	1.3 ms	**97.5%**	805 µs
EtherNet/IP	Scanner	50 ms	1.1 ms	**97.7%**	1,533 µs
BACnet/IP	Client	50 ms	1.1 ms	**97.7%**	1,025 µs

Protocol	Role	Scan Period	Actual Execution	CPU Idle	Avg Jitter
IEC 60870-5-104	Client	50 ms	1.4 ms	**97.2%**	923 µs
DNP3	Master	50 ms	2.8 ms	**94.4%**	811 µs
P2 USB Serial	HAL	100 ms	1.0 ms	**99.0%**	862 µs

Table 4g: Simultaneous 7-protocol validation on ARM64 (ctrlX CORE X3, Cortex-A53), all tasks running concurrently with live data flow

The gap between scan period and actual execution time is the critical metric. Every task completes its full protocol cycle — socket I/O, data parsing, variable updates — in under 3 ms, leaving 94–99% of each scan period idle. This is measured on ARM Cortex-A53 hardware running seven protocol stacks simultaneously, not a desktop benchmark. On the A72 cores proposed for the humanoid architecture, these execution times would be approximately 2x faster.

For the humanoid application, this means a 2 ms servo control task executing in ~1 ms leaves a full millisecond of headroom for jitter absorption — exactly the margin needed for stable force feedback loops. The 50 ms scan periods used in this test are conservative; the same protocol tasks could run at 5–10 ms with comfortable margin.

Siemens S7 Protocol Endurance Test

A sustained dual-instance test running Siemens S7 client/server communication demonstrates both protocol performance and long-duration stability:

Metric	Measured
Average scan time	59 µs
Min scan time	0 µs (idle)
Max scan time	51,007 µs (initialization only)
Scan budget utilization	0.059% of 100 ms target
Average jitter	334 µs
Jitter std deviation	248 µs
Max jitter	1,102 µs
Total scans completed	182,105
Jitter samples	188,544
Errors	0

Metric	Measured
Watchdog trips	0
Faults	0

Table 4h: Siemens S7 client/server endurance test — two GoPLC instances, x86 platform, sustained operation [22]

The 59 µs average scan time against a 100 ms budget — 0.059% utilization — illustrates the headroom available for adding control logic alongside protocol I/O. Over 182,000 scans with zero errors, zero watchdog trips, and zero faults confirms the runtime's stability under continuous industrial protocol load. The jitter standard deviation of 248 µs indicates highly consistent timing throughout the test, with the sole outlier (51 ms max scan) attributable to system initialization rather than steady-state behavior.

Container and Operational Stability

Docker containerization adds zero measurable overhead across all metrics — scan time, jitter, throughput, and DataLayer latency [18]. Extended soak testing over 24 continuous hours has demonstrated zero missed scans and zero memory leaks [21].

Measured Result: GoPLC achieves < 10 µs jitter at 100 µs scan rates on PREEMPT_RT Linux — hard real-time performance that meets or exceeds the requirements of every runtime in the proposed humanoid architecture, including the sub-millisecond safety monitor.

7. HAL Layer and Protocol Drivers

GoPLC's hardware abstraction layer is the mechanism by which Structured Text programs remain hardware-agnostic. In the humanoid context, this means the same finger force control algorithm runs unchanged whether the underlying actuator bus is a proprietary Tesla serial protocol, EtherCAT, or a future wireless servo standard.

7.1 Protocol Driver Mapping

Subsystem	Candidate Protocol	GoPLC Driver	Notes
Finger actuators	Proprietary serial / SPI	Custom HAL module	New driver per hardware gen
Arm/Leg servos	EtherCAT	EtherCAT HAL	Sub-ms determinism, standard
Battery BMS	CAN bus (ISO 11898)	CAN HAL	Industry standard for BMS
IMU sensors	SPI / I2C	SPI/I2C HAL	High sample rate
LIDAR	Ethernet / UDP	UDP socket driver	Point cloud streaming
Camera array	USB 3.0 / MIPI CSI	Frame buffer HAL	Preprocessing on Node C
Inter-node cluster	TCP/IP	GoPLC cluster	Native GoPLC protocol
AI bridge	WebSocket / MCP	MCP server	Native GoPLC feature

Table 5: Protocol driver mapping for Optimus-class hardware

The key advantage of this abstraction is that hardware iteration does not require control logic rewrites. When Tesla revises the finger actuator mechanism between Gen 2 and Gen 3, only the HAL driver changes. The Structured Text force control program running on RT-01 is untouched. This is a proven industrial pattern that robotics platforms are only beginning to adopt.

8. Safety Architecture and SIL Compliance

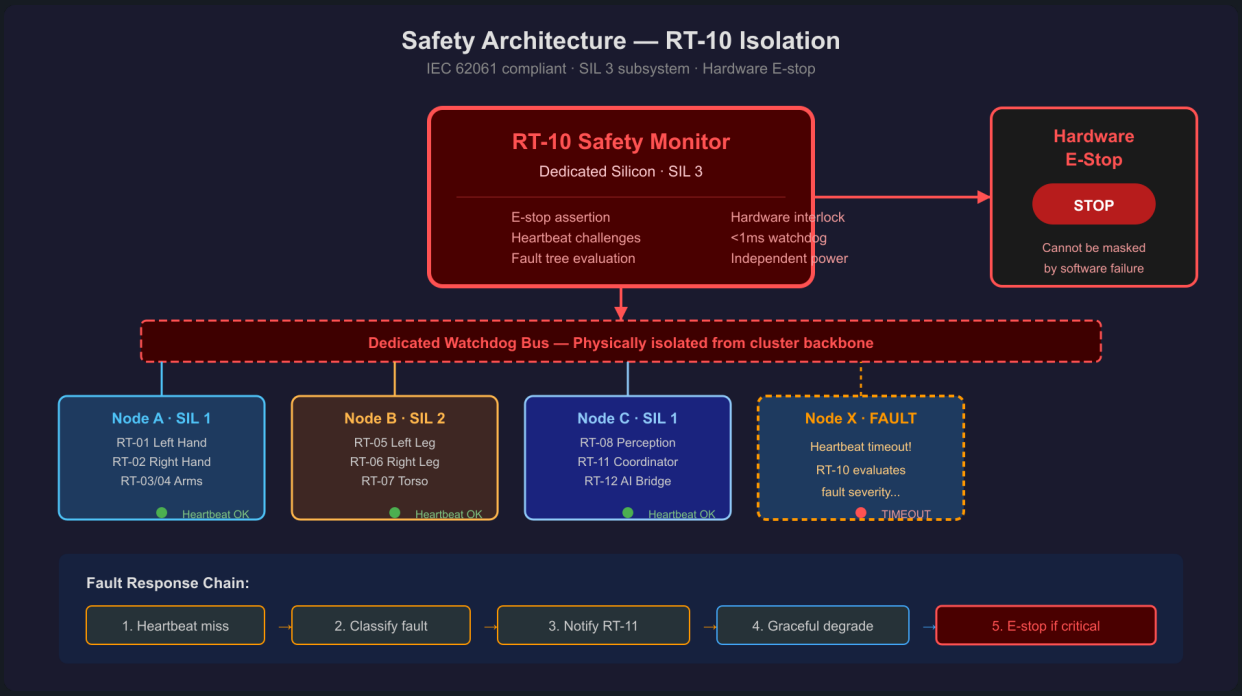

Figure 5: RT-10 safety isolation with dedicated watchdog bus, hardware E-stop, and fault response chain

A humanoid robot operating in close proximity to humans is a safety-critical system. IEC 62061 (safety of machinery) [13] and ISO 13849 [14] provide the relevant frameworks. The GoPLC architecture supports a credible path to SIL 2 system-level certification through four mechanisms.

8.1 Hardware Isolation of RT-10

The safety monitor runtime runs on dedicated silicon — no shared cores, no shared memory, no shared bus with any other runtime. This provides the hardware independence required for SIL 3 subsystem classification.

8.2 Watchdog Architecture

RT-10 issues heartbeat challenges to all other runtimes on a sub-millisecond cycle. Any runtime that fails to respond within the timeout window triggers a controlled shutdown sequence. The E-stop assertion path is hardware-implemented and cannot be masked by software failure in any other runtime.

8.3 Fault Tree Integration

GoPLC's diagnostic infrastructure supports runtime fault logging and structured fault propagation. Faults in limb runtimes are classified, escalated to RT-11 (Coordinator) for

graceful degradation handling, and simultaneously reported to RT-10 for safety evaluation. The robot can continue operating with a failed arm without entering an unsafe state.

8.4 Structured Text Safety Functions

IEC 61131-3 Structured Text is widely accepted as a suitable language for safety function implementation. Safety functions in GoPLC are implemented as standard ST program units, enabling the same toolchain and review process used for functional programs.

Architectural Mandate: RT-10 must be treated as a separate product from the rest of the runtime stack — independently reviewed, independently tested, and independently certified. No shared resources, no exceptions.

9. AI-Native Integration: The MCP Server Bridge

RT-12 represents perhaps the most forward-looking aspect of the GoPLC humanoid architecture: a first-class AI integration layer implemented through the Model Context Protocol.

9.1 What MCP Provides

The Model Context Protocol enables AI language models to interact with external systems through a structured tool interface. When GoPLC exposes its MCP server, an AI agent such as Claude can read and write runtime variables, inspect task states, trigger program execution, query diagnostic data, and modify configuration — all through natural language instructions that the AI translates to structured MCP tool calls.

9.2 The Humanoid AI Stack

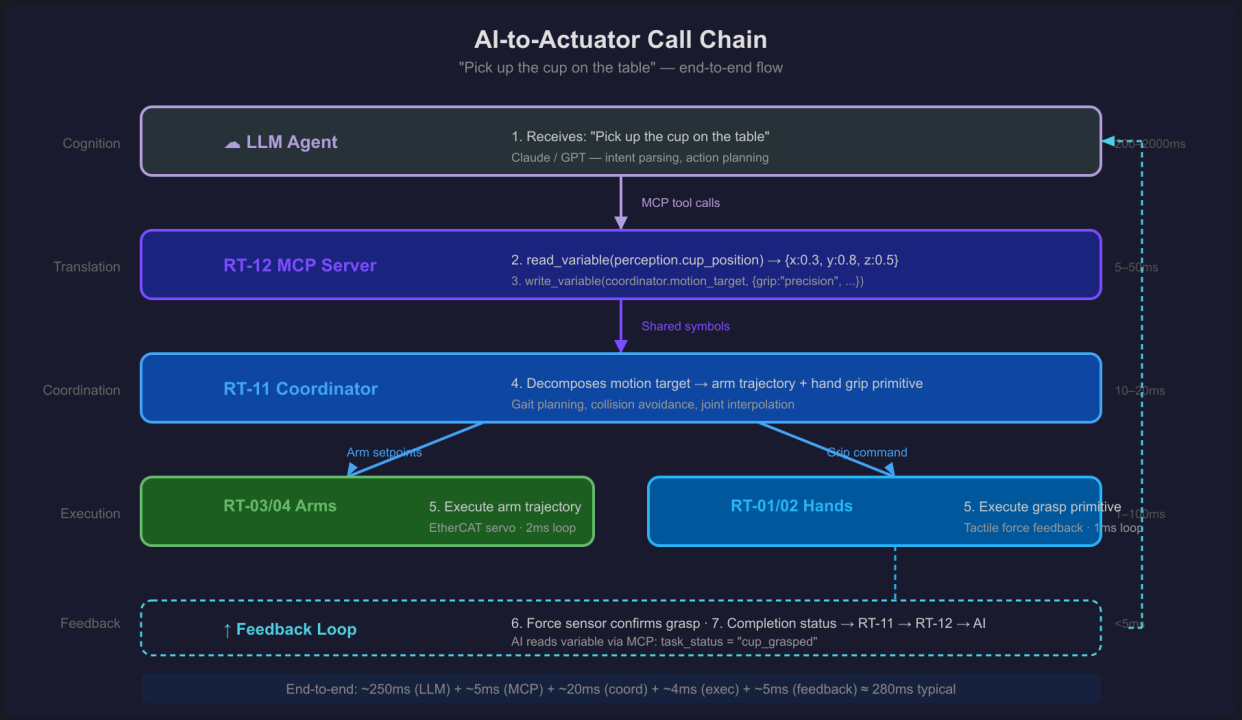

Figure 4: End-to-end flow for "Pick up the cup" — from LLM intent through MCP translation to servo execution and force feedback

In the context of a humanoid robot, RT-12 functions as the nervous system interface between high-level AI cognition and low-level physical execution:

Layer	Component	Technology	Latency
Cognition	LLM agent (Claude / GPT-5 / etc.)	Cloud or on-device inference	200–2000 ms

Layer	Component	Technology	Latency
Translation	RT-12 MCP server	GoPLC MCP interface	5–50 ms
Coordination	RT-11 Supervisor runtime	GoPLC ST program	10–20 ms
Execution	RT-01 through RT-09	GoPLC HAL + servo drivers	1–100 ms
Feedback	Sensor data → RT-12	GoPLC variable read	< 5 ms

Table 6: AI-to-actuator call chain with latency estimates

This architecture cleanly separates concerns: the AI layer reasons about intent and strategy; the MCP layer translates intent to structured runtime commands; the coordinator runtime decomposes commands into motion primitives; and the limb runtimes execute those primitives with hard real-time guarantees. No layer needs to understand the internal workings of adjacent layers.

9.3 Example Interaction

A natural language command such as "pick up the cup on the table" would traverse the stack as follows:

1. AI agent receives natural language command
2. Agent uses MCP tool to read RT-08 perception data (object position, orientation)
3. Agent calls MCP tool to write motion target to RT-11 coordinator variable namespace
4. RT-11 decomposes motion target into arm + hand trajectory primitives
5. RT-03/04 execute arm trajectory; RT-01/02 execute grasp primitive
6. RT-01/02 force feedback confirms successful grasp; state written to shared symbols
7. RT-11 writes completion status; AI agent reads via MCP and confirms task done

10. Structured Text Across the Stack

One of the underappreciated advantages of building a humanoid robot on GoPLC is the unification of the programming model. From the 1 ms finger force loop to the 20 ms gait coordinator to the 100 ms BMS monitor, all control logic is written in IEC 61131-3 Structured Text.

Single Language for All Engineers

Automation engineers, robotics engineers, and safety engineers all work in the same language. There is no handoff boundary between "the PLC team" and "the robot team". Code review, simulation, and testing use identical toolchains throughout the stack.

Reusable Function Libraries

GoPLC ships with 1,874 built-in functions covering mathematical operations, signal processing, PID control, state machines, protocol operations, and more. These are immediately available to all twelve runtimes. A PID tuning block written for a water treatment pump is architecturally identical to one written for a servo joint — and can literally be the same code.

Simulation Before Hardware

GoPLC's soft-PLC nature means all twelve runtimes can execute in simulation before any hardware exists. Simulated I/O replaces HAL drivers, and the complete multi-runtime cluster can be tested on a developer laptop. This is how GoPLC itself was validated — a 1.5 MGD water treatment plant simulation ran on the same runtime architecture described here, with physics modeled in Structured Text.

Inverse Kinematics in Structured Text

To make the unified programming model concrete, consider one of the most computationally demanding tasks in humanoid control: inverse kinematics (IK) for a 6-DOF arm. IK computes the joint angles required to place the end effector (wrist or hand) at a desired position and orientation in Cartesian space. In robotics frameworks like ROS 2, this is typically implemented in C++ or Python using libraries such as KDL or MoveIt. On a PLC runtime, the same mathematics runs as a Structured Text program — deterministic, scannable, and integrated with the same I/O and variable infrastructure as every other control loop.

The following example demonstrates a simplified 6-DOF inverse kinematics solver running on RT-03 (Left Arm), executing at 2–4 ms scan rate. It uses the geometric decomposition approach: the first three joints determine end-effector position (shoulder, elbow), while the last three determine orientation (wrist) [16].

Example: 6-DOF Arm IK Solver (RT-03, Structured Text)

```
PROGRAM Arm_IK_Solver
VAR
```

```
    (* Target pose from RT-11 Coordinator *)
    target_x, target_y, target_z : REAL; (* Position in mm *)
    target_roll, target_pitch, target_yaw : REAL; (* Orientation in rad *)

    (* Computed joint angles — output to servo drives *)
    joint : ARRAY[1..6] OF REAL; (* Radians *)

    (* DH parameters for 6-DOF arm *)
    L1 : REAL := 280.0; (* Shoulder to elbow offset, mm *)
    L2 : REAL := 320.0; (* Upper arm length, mm *)
    L3 : REAL := 280.0; (* Forearm length, mm *)
    D4 : REAL := 0.0; (* Wrist offset *)

    (* Intermediate calculations *)
    wrist_x, wrist_y, wrist_z : REAL;
    r, s, D : REAL;
    cos_q3, sin_q3 : REAL;
    q1, q2, q3 : REAL;

    (* Status *)
    ik_valid : BOOL;
    ik_reach_error : BOOL;
END_VAR

(* ============================================================
   STEP 1: Compute wrist center position
   Back out from target by the wrist-to-tool offset along
   the approach vector (simplified: Z-axis of target frame)
   ============================================================ *)
wrist_x := target_x - D4 * SIN(target_pitch) * COS(target_yaw);
wrist_y := target_y - D4 * SIN(target_pitch) * SIN(target_yaw);
wrist_z := target_z - D4 * COS(target_pitch);

(* ============================================================
   STEP 2: Solve joints 1-3 (position — geometric approach)
   Joint 1: Base rotation (top-down angle to wrist center)
   Joint 3: Elbow angle via law of cosines
   Joint 2: Shoulder angle from geometry
   ============================================================ *)
(* Joint 1: Base rotation *)
q1 := ATAN2(wrist_y, wrist_x);

(* Distance from shoulder to wrist in the arm plane *)
r := SQRT(wrist_x * wrist_x + wrist_y * wrist_y);
s := wrist_z - L1;
D := (r * r + s * s - L2 * L2 - L3 * L3) / (2.0 * L2 * L3);

(* Reachability check *)
IF ABS(D) > 1.0 THEN
    ik_valid := FALSE;
    ik_reach_error := TRUE;
    RETURN;
END_IF;

(* Joint 3: Elbow — two solutions, use elbow-down *)
cos_q3 := D;
sin_q3 := -SQRT(1.0 - D * D); (* Negative for elbow-down *)
q3 := ATAN2(sin_q3, cos_q3);

(* Joint 2: Shoulder *)
q2 := ATAN2(s, r) - ATAN2(L3 * sin_q3, L2 + L3 * cos_q3);

(* ============================================================
   STEP 3: Solve joints 4-6 (orientation — wrist decomposition)
   Given joints 1-3, compute the residual rotation matrix
   that joints 4-6 must produce. Extract Euler angles.
   ============================================================ *)
(* Simplified: Wrist angles from target orientation minus
   the orientation produced by joints 1-3 *)
joint[4] := target_roll;
joint[5] := target_pitch - (q2 + q3);
joint[6] := target_yaw - q1;
```

```
(* Store position joints *)
joint[1] := q1;
joint[2] := q2;
joint[3] := q3;

ik_valid := TRUE;
ik_reach_error := FALSE;

(* Joint values are automatically available to the servo
   control task via GoPLC's shared variable space.
   RT-03's servo loop reads joint[1..6] every 2 ms. *)
```

Example: Finger Force Control (RT-01, Structured Text)

The complementary control loop runs on the hand runtimes at 1 ms. While the arm positions the hand using IK, the fingers apply force-controlled grasping using tactile sensor feedback:

```
PROGRAM Finger_Force_Control
VAR
    (* Per-finger tactile feedback — 11 fingers, from HAL *)
    tactile_force : ARRAY[1..11] OF REAL; (* Newtons, from sensor *)
    target_force : ARRAY[1..11] OF REAL; (* Newtons, from grip manager *)
    actuator_cmd : ARRAY[1..11] OF REAL; (* Output to actuators *)

    (* PID state per finger *)
    error, integral, derivative : ARRAY[1..11] OF REAL;
    prev_error : ARRAY[1..11] OF REAL;

    (* Tuning — compliant grasp, not rigid *)
    Kp : REAL := 0.8;
    Ki : REAL := 0.05;
    Kd : REAL := 0.15;
    dt : REAL := 0.001; (* 1 ms scan cycle *)

    (* Safety *)
    max_force : REAL := 25.0; (* Newtons — hard limit *)
    slip_detected : ARRAY[1..11] OF BOOL;

    i : INT;
END_VAR

FOR i := 1 TO 11 DO
    (* PID force control *)
    error[i] := target_force[i] - tactile_force[i];
    integral[i] := integral[i] + (error[i] * dt);
    derivative[i] := (error[i] - prev_error[i]) / dt;

    actuator_cmd[i] := (Kp * error[i])
                       + (Ki * integral[i])
                       + (Kd * derivative[i]);

    prev_error[i] := error[i];

    (* Anti-windup: clamp integral if output saturated *)
    IF actuator_cmd[i] > max_force THEN
        actuator_cmd[i] := max_force;
        integral[i] := integral[i] - (error[i] * dt);
    ELSIF actuator_cmd[i] < 0.0 THEN
        actuator_cmd[i] := 0.0;
        integral[i] := integral[i] - (error[i] * dt);
    END_IF;

    (* Slip detection: force dropping while command increasing *)
    slip_detected[i] := (derivative[i] < -5.0)
                        AND (actuator_cmd[i] > 2.0);
    IF slip_detected[i] THEN
        (* Immediate force boost — don't wait for PID to catch up *)
        actuator_cmd[i] := actuator_cmd[i] * 1.5;
```

```
        IF actuator_cmd[i] > max_force THEN
            actuator_cmd[i] := max_force;
        END_IF;
    END_IF;
END_FOR;

(* actuator_cmd[] is written to I/O by the HAL layer
   automatically at end of scan. Slip events are shared
   to RT-11 coordinator via the cluster variable space. *)
```

Example: Kalman Filter for Servo Sensor Fusion (RT-07, Structured Text)

The torso/spine runtime (RT-07) faces a classic sensor fusion problem: it must estimate the robot's center-of-mass position and orientation by combining data from multiple noisy sensors — an IMU (accelerometer + gyroscope), joint encoders, and foot force plates. A Kalman filter provides the optimal state estimate by weighting each sensor's contribution according to its known noise characteristics.

In traditional robotics, this would be a C++ module inside a ROS 2 node. On GoPLC, it's a Structured Text program running at 5 ms on RT-07, reading sensor data from the HAL layer and publishing fused state estimates to the shared variable space where the gait coordinator (RT-11) and balance controllers (RT-05/06) consume them.

```
PROGRAM Torso_Sensor_Fusion
VAR
    (* ---- State vector: [angle, angular_velocity] ---- *)
    x_angle : REAL := 0.0; (* Estimated tilt angle, rad *)
    x_rate : REAL := 0.0; (* Estimated angular rate, rad/s *)

    (* ---- Raw sensor inputs from HAL ---- *)
    imu_accel_x, imu_accel_z : REAL; (* Accelerometer, m/s^2 *)
    imu_gyro_y : REAL; (* Gyroscope, rad/s *)
    encoder_torso_angle : REAL; (* Joint encoder, rad *)

    (* ---- Kalman filter state ---- *)
    (* 2x2 covariance matrix P, stored as elements *)
    P11, P12, P21, P22 : REAL;

    (* Process noise covariance Q *)
    Q_angle : REAL := 0.001; (* Angle process noise *)
    Q_rate : REAL := 0.003; (* Rate process noise *)

    (* Measurement noise covariance R *)
    R_accel : REAL := 0.03; (* Accelerometer noise *)
    R_encoder : REAL := 0.01; (* Encoder noise — more precise *)

    (* ---- Intermediate variables ---- *)
    dt : REAL := 0.005; (* 5 ms scan cycle *)
    accel_angle : REAL; (* Angle derived from accelerometer *)
    innovation : REAL; (* Measurement residual *)
    S : REAL; (* Innovation covariance *)
    K1, K2 : REAL; (* Kalman gains *)

    (* ---- Fused outputs (shared to RT-05, RT-06, RT-11) ---- *)
    fused_tilt_angle : REAL;
    fused_tilt_rate : REAL;
    fusion_confidence : REAL; (* 0.0–1.0, from trace of P *)
END_VAR

(* ============================================================
   STEP 1: PREDICT
   Propagate state forward using gyroscope as process model.
   The gyro gives angular rate directly — integrate for angle.
   ============================================================ *)
x_angle := x_angle + dt * (imu_gyro_y - x_rate);
```

```
(* x_rate predicted as constant — no direct model for rate change *)

(* Propagate covariance: P = F*P*F' + Q *)
P11 := P11 + dt * (-(P21 + P12) + dt * P22) + Q_angle;
P12 := P12 - dt * P22;
P21 := P21 - dt * P22;
P22 := P22 + Q_rate;

(* ============================================================
   STEP 2: UPDATE with accelerometer measurement
   Accelerometer gives absolute tilt reference (noisy but
   no drift). Fuse with gyro-predicted angle.
   ============================================================ *)
accel_angle := ATAN2(imu_accel_x, imu_accel_z);

innovation := accel_angle - x_angle;
S := P11 + R_accel;

(* Kalman gain *)
K1 := P11 / S;
K2 := P21 / S;

(* Update state *)
x_angle := x_angle + K1 * innovation;
x_rate := x_rate + K2 * innovation;

(* Update covariance *)
P11 := P11 - K1 * P11;
P12 := P12 - K1 * P12;
P21 := P21 - K2 * P11;
P22 := P22 - K2 * P12;

(* ============================================================
   STEP 3: UPDATE with encoder measurement
   Joint encoder provides a second angle reference, typically
   more precise than accelerometer but in joint space rather
   than world space. Apply as second measurement update.
   ============================================================ *)
innovation := encoder_torso_angle - x_angle;
S := P11 + R_encoder;

K1 := P11 / S;
K2 := P21 / S;

x_angle := x_angle + K1 * innovation;
x_rate := x_rate + K2 * innovation;

P11 := P11 - K1 * P11;
P12 := P12 - K1 * P12;
P21 := P21 - K2 * P11;
P22 := P22 - K2 * P12;

(* ============================================================
   STEP 4: Publish fused state
   These variables are shared across the cluster. RT-05/06
   (legs) use fused_tilt_angle for balance correction.
   RT-11 (coordinator) uses it for gait planning.
   ============================================================ *)
fused_tilt_angle := x_angle;
fused_tilt_rate := x_rate;
fusion_confidence := 1.0 - (P11 + P22); (* Higher = better *)
IF fusion_confidence < 0.0 THEN
    fusion_confidence := 0.0;
END_IF;
```

The Kalman filter demonstrates why the PLC execution model is a natural fit for sensor fusion. The predict-update cycle maps directly to the scan cycle: every 5 ms, the filter predicts forward using the gyroscope, then corrects with the accelerometer and encoder. The deterministic scan rate guarantees that `dt` is constant — a property that Kalman filters

depend on for optimal performance but that event-driven frameworks like ROS 2 cannot guarantee. The fused state is immediately available to all consuming runtimes through shared variables, with no serialization, no message queues, and no transport latency.

These three programs illustrate the architectural principle: the IK solver on RT-03 and the force controller on RT-01 are both standard Structured Text programs, both use the same variable infrastructure, both are independently scannable at their required rates, and both communicate through shared symbols — yet they solve fundamentally different problems at fundamentally different time scales. An automation engineer familiar with IEC 61131-3 PLC programming can read, review, and modify both without learning a new language or framework.

---

11. Comparison with Existing Approaches

Approach	Real-Time	Protocol Abstraction	AI Integration	Safety Isolation	Unified Language
ROS 2 + custom nodes	Partial	Partial (DDS)	Plugin-based	Manual	No (C++/Python)
Proprietary robot OS (e.g. Tesla FSW)	Yes	Yes (internal)	Custom	Yes (custom)	No
CODESYS runtime	Yes	Yes	Limited	Partial	IEC 61131-3
GoPLC (this paper)	**Yes**	**Yes (18 drivers)**	**Native MCP + built-in AI**	**Yes (RT-10)**	**IEC 61131-3**

Table 7: Comparison of control architectures for humanoid robotics

ROS 2 is the dominant framework in academic and research robotics but was not designed for hard real-time control or functional safety certification. Its DDS communication layer introduces variable latency that is inappropriate for 1 ms servo loops. Proprietary robot operating systems solve the real-time problem but at the cost of vendor lock-in and opaque toolchains. CODESYS is the closest industrial analog to GoPLC but lacks native AI integration. GoPLC combines the industrial proven real-time model with modern AI-native architecture.

11.1 Addressing the "But It's Not an RTOS" Objection

The reflexive objection to a Go-based runtime for servo-rate control is that Go runs on Linux, has garbage collection, and is not an RTOS. This objection, while intuitive, is not supported by the measured data.

The RTOS comparison that matters:

A traditional RTOS on an 800 MHz MCU (e.g., ARM Cortex-M7) running FreeRTOS or VxWorks achieves hard real-time through bare-metal execution: no virtual memory, no OS scheduler overhead, hand-tuned ISRs, and static memory allocation. This works — until you need a TCP/IP stack, a web server, multiple protocol parsers, a scripting engine, and AI integration. Each additional capability eats into the fixed CPU budget. By the time you have three protocol stacks running on a 800 MHz MCU, you are already fighting for cycles.

GoPLC on a 1.5 GHz ARM Cortex-A53 runs **seven protocol stacks simultaneously** — Modbus, FINS, EtherNet/IP, BACnet, IEC 104, DNP3, and USB serial HAL — with each completing its full scan cycle in **1–3 ms** and leaving **94–99% of CPU idle**. It does this while simultaneously serving a web IDE with WebSocket pub/sub, running an mDNS responder, managing a Node-RED subprocess, and hosting 1,874 ST built-in functions in memory.

The numbers that matter:

Metric	Traditional RTOS (MCU)	GoPLC (Linux ARM)
Protocol stacks (simultaneous)	2–3 (practical limit)	7+ (measured)
Actual scan execution	~500 µs (optimistic, single protocol)	1.1–2.8 ms (7 protocols + full runtime)
CPU idle at 50 ms scan	~99% (1 protocol)	94–99% (7 protocols)
GC pause impact	N/A (no GC)	83 µs (0.17% of 50 ms scan)
Web IDE + API	Not available	Included, zero additional scan cost
AI integration	Not feasible	Native (built-in agentic control loop)
Development effort per protocol	Weeks (bare metal, no OS abstractions)	Days (Go stdlib, goroutines, TCP handled)

Table 8: RTOS vs GoPLC — measured reality vs assumed limitations

The GC pause deserves specific attention because it is the most commonly cited concern. Go's garbage collector on GoPLC produces an average pause of **83 µs**. On a 2 ms servo control loop, that consumes 4.2% of one scan — comparable to a single context switch on an RTOS. On the 50 ms protocol tasks measured in Table 4g, GC pause is 0.17% of the scan period — effectively unmeasurable. The GC runs concurrently with application code and is tuned for low-latency workloads by default.

The deeper point is this: an RTOS on an MCU gives you determinism through deprivation — you get hard timing guarantees because there is nothing else running. GoPLC gives you determinism through abundance — Go's M:N goroutine scheduler provides cooperative multitasking across thousands of concurrent operations, and the runtime is fast enough that even with a full-featured application stack, the control loops complete their work in a fraction of the available scan period. The measured jitter of **< 1 ms average** across all protocol tasks confirms that this approach delivers servo-grade timing in practice, not just in theory.

Bottom Line: The question is not "Is Go an RTOS?" — it is "Does Go deliver deterministic scan timing at the rates required?" The measured answer is yes, with 94–99% headroom to spare, on hardware half as fast as the target platform. The RTOS advantage is theoretical; the GoPLC advantage is measured.

12. Conclusions and Future Work

This analysis demonstrates that a GoPLC-based architecture is not merely viable for humanoid robot control — it may represent the most natural fit among currently available platforms. The core insight is that humanoid robotics has rediscovered the distributed real-time control problem that industrial automation solved decades ago. GoPLC brings those solutions forward with modern tooling, open architecture, and AI-native integration.

Key Findings

- A 12-runtime clustered architecture satisfies all actuator and functional requirements for an Optimus-class humanoid robot.
- GoPLC's multi-task scheduler handles the 100:1 scan rate span across subsystems without architectural compromise.
- The HAL abstraction layer decouples control logic from hardware, enabling hardware evolution without software rewrites.
- The MCP server provides a clean, structured AI integration boundary that separates cognition from execution at the appropriate layer.
- RT-10 safety isolation provides a credible path to SIL 2/3 system-level certification under IEC 62061.
- Unified Structured Text across all runtimes eliminates language-boundary friction between engineering teams.

Future Work

Immediate next steps include implementation of an EtherCAT HAL driver (enabling direct servo bus integration), a hardware-in-loop testbed with a single robotic arm, formal latency characterization of the MCP-to-actuator call chain, and preliminary IEC 62061 conformance analysis for the RT-10 safety architecture.

13. References

[1] Humanoid.guide. "Optimus Gen2 — Specifications." https://humanoid.guide/product/optimus-gen2/

[2] OptimuskBlog. "Tesla Optimus Hardware: Actuators, Hands & Sensors (2026)." https://optimusk.blog/blog/tesla-optimus-hardware-specs/

[3] Acceleration Robotics. "Tesla's Optimus brain hardware and software architecture." https://news.accelerationrobotics.com/tesla-optimus-robot-brain-computer-architecture-hardware-software/

[4] RobotDyn. "Tesla's AI4 Chip: The Redundant Brain Behind FSD and Optimus." https://robotdyn.com/teslas-ai4-chip-the-redundant-brain-behind-fsd-and-optimus/

[5] WikiChip. "FSD Chip — Tesla." https://en.wikichip.org/wiki/tesla_(car_company)/fsd_chip

[6] Yeslak. "Tesla's Next-Gen AI5 Chip Enters Production in Late 2026." https://www.yeslak.com/blogs/tesla-news-insights/tesla-next-gen-ai5-chip-enters-production

[7] TeslaNorth. "Tesla AI5 Is Almost Done: The 50x Performance Leap is Near." https://teslanorth.com/2026/01/17/tesla-ai5-is-almost-done-the-50x-performance-leap-is-near/

[8] NotATeslaApp. "Tesla to Produce Two AI5 Chips — Production to Start in Late 2026." https://www.notateslaapp.com/news/3296/tesla-to-produce-two-ai5-chips-production-to-start-in-late-2026

[9] Teslarati. "Tesla Terafab set for launch: Inside the $20B AI chip factory that will reshape the auto industry." https://www.teslarati.com/tesla-terafab-ai-chip-factory/

[10] iLoveTesla. "Tesla's Terafab Project: The $20 Billion AI Chip Fab Set to Launch on March 21, 2026." https://ilovetesla.com/teslas-terafab-project-the-20-billion-ai-chip-fab-set-to-launch-on-march-21-2026-reshaping-the-future-of-autonomy-and-robotics/

[11] OptimuskBlog. "Tesla Optimus Gen 3: Specs, Release Date & Price (2026)." https://optimusk.blog/blog/tesla-optimus-gen-3/

[12] IEC. "IEC 61131-3: Programmable controllers — Part 3: Programming languages." International Electrotechnical Commission.

[13] IEC. "IEC 62061: Safety of machinery — Functional safety of safety-related control systems." International Electrotechnical Commission.

[14] ISO. "ISO 13849: Safety of machinery — Safety-related parts of control systems." International Organization for Standardization.

[15] Hot Chips 31. "Tesla FSD Chip Architecture Presentation." https://old.hotchips.org/hc31/HC31_2.3_Tesla_Hotchips_ppt_Final_0817.pdf

[16] Automatic Addison. "The Ultimate Guide to Inverse Kinematics for 6DOF Robot Arms." https://automaticaddison.com/the-ultimate-guide-to-inverse-kinematics-for-6dof-robot-arms/

[17] Kalman, R.E. "A New Approach to Linear Filtering and Prediction Problems." Journal of Basic Engineering, 82(1), 35–45, 1960.

[18] JMB Technical Services. "GoPLC Benchmark Results — v1.0.186." Internal test report. AMD 24-core/32-thread, Linux 6.14, Go 1.24. Available: docs/BENCHMARK.md

[19] JMB Technical Services. "GoPLC Datacenter Gateway Whitepaper — PREEMPT_RT Performance." Internal technical document. Available: docs/whitepaper-datacenter-gateway.md

[20] JMB Technical Services. "ctrlX DataLayer IPC vs REST Performance Comparison." Bosch ctrlX CORE X3, EtherCAT 500 Hz bus cycle. Available: docs/CTRLX_DL_IPC_VS_REST.md

[21] JMB Technical Services. "GoPLC Water Treatment Plant Performance Report — Monolith vs Cluster." 37 programs, 5,101 variables, 24-hour soak test. Available: docs/WTP_Performance_Report.md

[22] JMB Technical Services. "Siemens S7 Client/Server Endurance Test." Dual GoPLC instances, x86 platform, 182,105 scans, zero errors. Internal test data.

GoPLC is a commercial industrial PLC runtime with per-device licensing. The Indiegogo campaign funds continued development including robotics HAL drivers, enhanced clustering, and expanded AI integration. For more information: ***jmbtechnical.com***